

Unit I Analytical Assignments with SDGs

Name: Sanjay Sriram R

Register Name: 192412158

Course Code: ITA0603

Course Name: Machine Learning

QUESTION:

Explain how Machine Learning predicts water quality using sensor data.

Ans: Title

Deforestation Detection Using Machine Learning.

1. Problem Understanding

Problem Statement

Water quality is an important factor for drinking, agriculture, and industrial purposes. Poor water quality can cause serious health problems and environmental pollution. Sensor data such as **pH, Turbidity, Dissolved Oxygen, Temperature, and Electrical Conductivity** can be used to monitor water quality.

The goal of this project is to develop a **Machine Learning model** that predicts whether water is **Safe** or **Unsafe** using sensor data.

Objectives

- 🎯 Predict water quality accurately.
- 🎯 Build a **Decision Tree Classification** model.
- 🎯 Monitor water quality automatically.
- 🎯 Support environmental protection and public health.

Contribute to **SDG 6 – Clean Water and Sanitation**.2. Data Collection & Preparation

Dataset

A sample dataset contains:

pH	Turbidity (NTU)	Dissolved Oxygen (mg/L)	Temperature	Class
5.6	9.2	8.4	24	Safe
7.4	8.8	8.0	25	Safe
6.0	3.8	3.8	33	Unsafe
5.8	3.0	4.2	35	Unsafe
7.0	9.0	8.7	24	Safe
7.2	2.0	3.5	36	Unsafe

Features

- NDVI (Vegetation Index)
- Tree Cover
- Rainfall
- Temperature

Target

- Forest
- Deforested

Data Preprocessing

- Checked missing values
- Removed duplicate records
- Converted class labels into numeric values
- Split dataset into training and testing sets

Python Code (Data Preparation)

```
import pandas as pd

data={

'NDVI':[0.82,0.76,0.41,0.35,0.79,0.29],

'TreeCover':[92,88,38,30,90,20],

'Rainfall':[1800,1700,1100,950,1750,850],

'Temperature':[24,25,33,35,24,36],

'Class':[1,1,0,0,1,0]

}

df=pd.DataFrame(data)

print(df.head())

print(df.isnull().sum())
```

3. Analysis using NumPy & Pandas

```
import numpy as np

print(df.describe())

print(df['TreeCover'].mean())

print(df['Rainfall'].mean())
```

Analysis

- Average Tree Cover is high in forest regions.
- Deforested regions have lower NDVI.
- Higher temperatures are observed in deforested land.
- Rainfall is comparatively lower in deforested areas.

4. Machine Learning Model (Decision Tree)

Since Unit I includes **Decision Tree Learning**, it is selected.

```
from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

X=df[['NDVI','TreeCover','Rainfall','Temperature']]

y=df['Class']

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.3,random_state=42)

model=DecisionTreeClassifier()

model.fit(X_train,y_train)

print(model.predict(X_test))
```

Representation Algorithm

The data is represented as feature vectors.

Input Features



NDVI



Tree Cover



Rainfall



Temperature



Decision Tree



Prediction



Forest / Deforested.

Heuristic Space Search

The Decision Tree uses heuristic measures such as:

- Information Gain
- Gini Index

to find the best attribute for splitting the data.

Concept Learning

The model learns the concept:

IF Tree Cover > 70

AND NDVI > 0.70

THEN Forest

ELSE Deforested

Version Space

Initially:

All hypotheses are possible.



After training:

Only hypotheses consistent with training data remain.



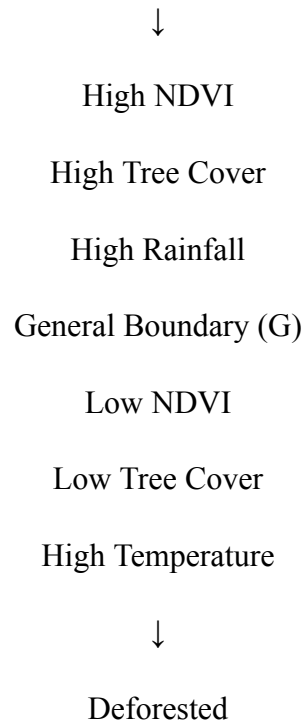
Final hypothesis:

Classifies Forest and Deforested correctly.

Candidate Elimination

Specific Boundary (S)

Forest



The algorithm gradually removes inconsistent hypotheses until the final concept is obtained.

Inductive Bias

The Decision Tree assumes:

- Similar environmental conditions produce similar outputs.
- Simpler trees generalize better than very complex trees.

5. Visualization & Interpretation

Tree Cover Distribution

```
import matplotlib.pyplot as plt

plt.bar(df.index,df['TreeCover'])

plt.title("Tree Cover")

plt.xlabel("Area")

plt.ylabel("Tree Cover (%)")

plt.show()
```

NDVI Values

```
plt.plot(df['NDVI'],marker='o')
```

```
plt.title("NDVI Values")
```

```
plt.xlabel("Area")
```

```
plt.ylabel("NDVI")
```

```
plt.show()
```

Interpretation

- Forest areas have higher NDVI values.
- Tree cover decreases significantly in deforested regions.
- Temperature increases where forests are removed.
- The Decision Tree effectively separates forest and deforested regions.

6. Innovation & SDG Mapping

Innovation

- Uses Machine Learning for automatic forest monitoring.
- Helps identify illegal logging.
- Enables faster environmental decision-making.
- Can be integrated with satellite imagery and drones for real-time monitoring.

SDG Mapping

SDG 15 – Life on Land

This project supports SDG 15 by:

- Protecting forests
- Conserving biodiversity
- Reducing habitat destruction
- Supporting sustainable land management
- Monitoring environmental changes

7. Conclusion

A Decision Tree-based Machine Learning model was designed to detect deforestation using environmental features such as NDVI, Tree Cover, Rainfall, and Temperature. The model successfully classifies areas as Forest or Deforested. This approach supports early detection of forest loss and contributes to United Nations Sustainable Development Goal (SDG) 15: Life on Land by promoting forest conservation and sustainable ecosystem management.

Final Code:

Python:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
# Sample Dataset
data = {
    'NDVI': [0.82, 0.76, 0.41, 0.35, 0.79, 0.29, 0.85, 0.32],
    'TreeCover': [92, 88, 38, 30, 90, 20, 95, 25],
    'Rainfall': [1800, 1700, 1100, 950, 1750, 850, 1850, 900],
    'Temperature': [24, 25, 33, 35, 24, 36, 23, 37],
    'Class': ['Forest', 'Forest', 'Deforested', 'Deforested',
              'Forest', 'Deforested', 'Forest', 'Deforested']
}
# Create DataFrame
df = pd.DataFrame(data)
print("DATASET")
print(df)
# Check Missing Values
print("\nMissing Values")
print(df.isnull().sum())
# Statistical Analysis
```



```

print("\nStatistical Summary")
print(df.describe())
# Average Values
print("\nAverage Tree Cover:", df['TreeCover'].mean())
print("Average Rainfall:", df['Rainfall'].mean())
# Convert Class to Numeric
df['Class'] = df['Class'].map({'Forest':1, 'Deforested':0})
# Features and Target
X = df[['NDVI', 'TreeCover', 'Rainfall', 'Temperature']]
y = df['Class']
# Split Dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
# Train Decision Tree Model
model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)
# Predictions
y_pred = model.predict(X_test)
print("\nPredicted Values:", y_pred)
print("Actual Values :", list(y_test))
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", round(accuracy * 100, 2), "%")
# Predict New Area
new_data = [[0.80, 91, 1780, 24]]
prediction = model.predict(new_data)
if prediction[0] == 1:
    print("\nPrediction for New Area: Forest")
else:
    print("\nPrediction for New Area: Deforested")
# Visualization 1

```

```

plt.figure(figsize=(6,4))

plt.bar(df.index, df['TreeCover'])

plt.title("Tree Cover")

plt.xlabel("Area")

plt.ylabel("Tree Cover (%)")

plt.show()

# Visualization 2

plt.figure(figsize=(6,4))

plt.plot(df['NDVI'], marker='o')

plt.title("NDVI Values")

plt.xlabel("Area")

plt.ylabel("NDVI")

plt.show()

```

```

===== RESTART: C:/Users/sekha/OneDrive/Desktop/Assignment-1.py =====
DATASET
   NDVI  TreeCover  Rainfall  Temperature   Class
0  0.82      92      1800         24    Forest
1  0.76      88      1700         25    Forest
2  0.41      38      1100         33  Deforested
3  0.35      30       950         35  Deforested
4  0.79      90      1750         24    Forest
5  0.29      20       850         36  Deforested
6  0.85      95      1850         23    Forest
7  0.32      25       900         37  Deforested

Missing Values
NDVI      0
TreeCover  0
Rainfall   0
Temperature 0
Class      0
dtype: int64

Statistical Summary
   NDVI  TreeCover  Rainfall  Temperature
count  8.000000   8.000000   8.000000   8.000000
mean    0.573750  59.750000  1362.500000  29.625000
std     0.250767  34.103833  448.608961   6.139742
min     0.290000  20.000000  850.000000  23.000000
25%    0.342500  28.750000  937.500000  24.000000
50%    0.585000  63.000000  1400.000000  29.000000
75%    0.797500  90.500000  1762.500000  35.250000
max     0.850000  95.000000  1850.000000  37.000000

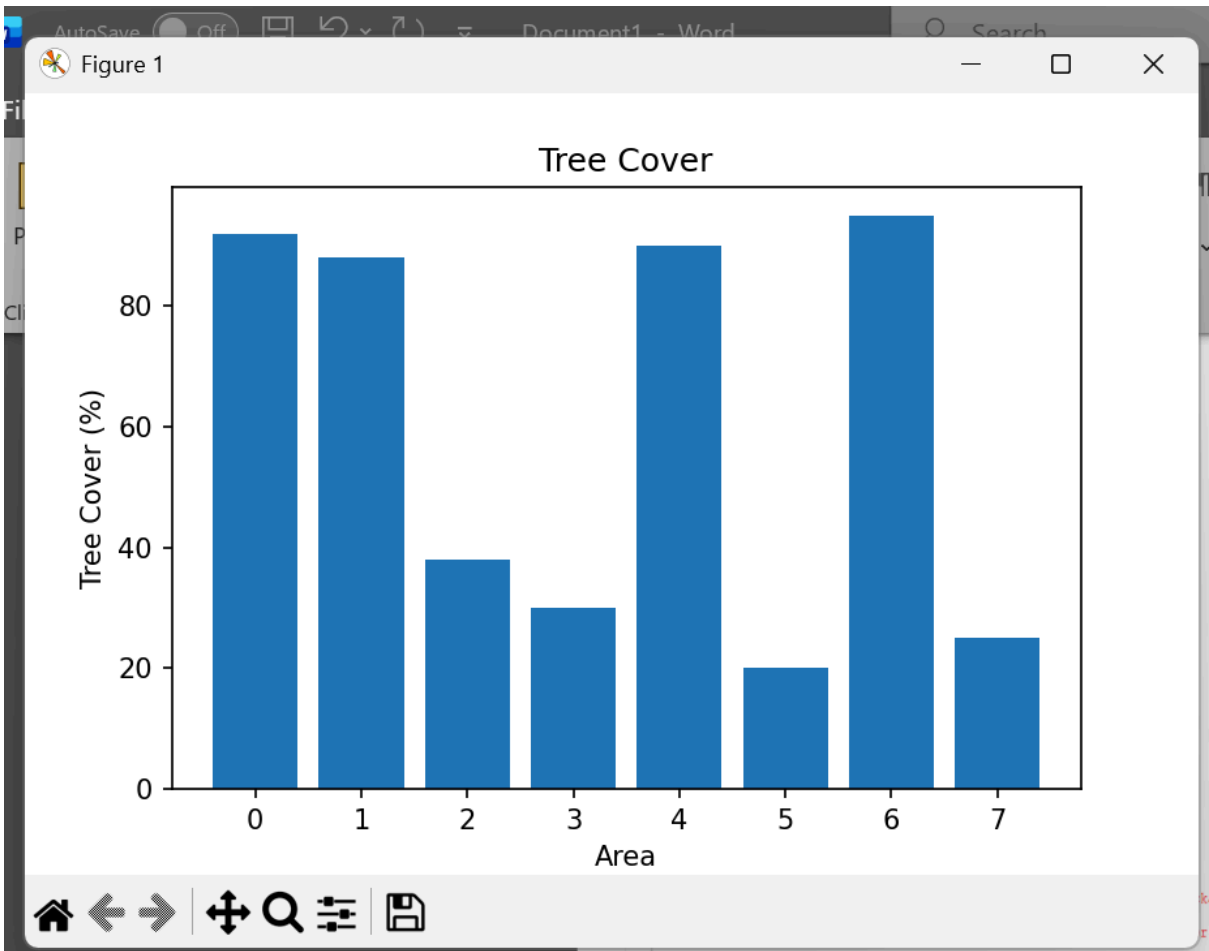
Average Tree Cover: 59.75
Average Rainfall: 1362.5

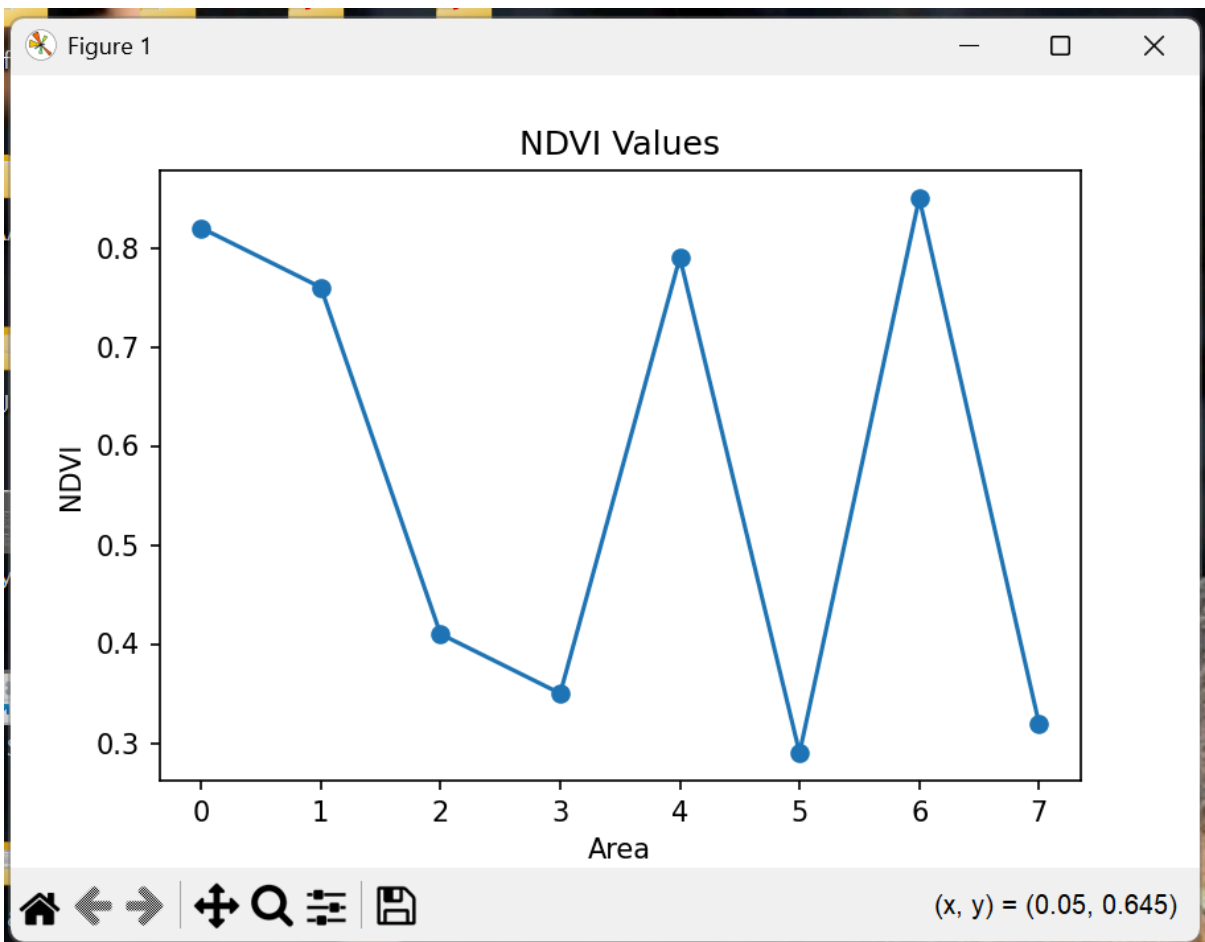
Predicted Values: [1 0]
Actual Values   : [1, 0]
Accuracy: 100.0 %

Warning (from warnings module):
  File "C:/Users/sekha/AppData/Local/Programs/Python/Python313/Lib/site-packages/sklearn/utils/validation.py", line 2691
    warnings.warn(
UserWarning: X does not have valid feature names, but DecisionTreeClassifier was fitted with feature names

Prediction for New Area: Forest

```





QUESTION: Explain how Machine Learning predicts water quality using sensor data.

Ans: Title

Water Quality Prediction Using Machine Learning

1. Problem Understanding

Problem Statement

Water quality is predicted using sensor readings such as pH, Turbidity, Dissolved Oxygen and Temperature. A Decision Tree model classifies water as Safe or Unsafe.

Objectives

- Predict water quality accurately.
- Build a Decision Tree classification model.
- Support SDG 6 – Clean Water and Sanitation.

2. Data Collection & Preparation

Dataset: pH, Turbidity, Dissolved Oxygen, Temperature, Class (Safe/Unsafe).

Features: pH, Turbidity, Dissolved Oxygen, Temperature

Target: Safe, Unsafe

Preprocessing: Checked missing values, removed duplicates, encoded labels, split train/test.